Task C — Execution (PID Brake + Alert)

Design Document

Jéssica Roberta de Souza Santos
April 2026

1 Overview

This document describes the design choices, data flow, and requirements traceability for the Execution module of the AEB system. The module comprises two source files: `aeb_pid.c` (PI brake controller) and `aeb_alert.c` (alert mapping and driver override detection).

The module implements **FR-BRK-001 through FR-BRK-007** (braking control) and **FR-ALR-001 through FR-ALR-004** (driver alerts), plus **FR-DEC-006** and **FR-DEC-007** (driver override conditions) from SRS v2.0. The implementation is derived from three Simulink charts: `chart_96` (PID_Logic), `chart_79` (Alert_Map), and `chart_88` (Override_Det).

2 Architecture and Data Flow

2.1 Position in the System

The Execution module sits in the **Controller thread** (priority 2, 10 ms cycle) and transforms FSM decisions into physical actuator commands:

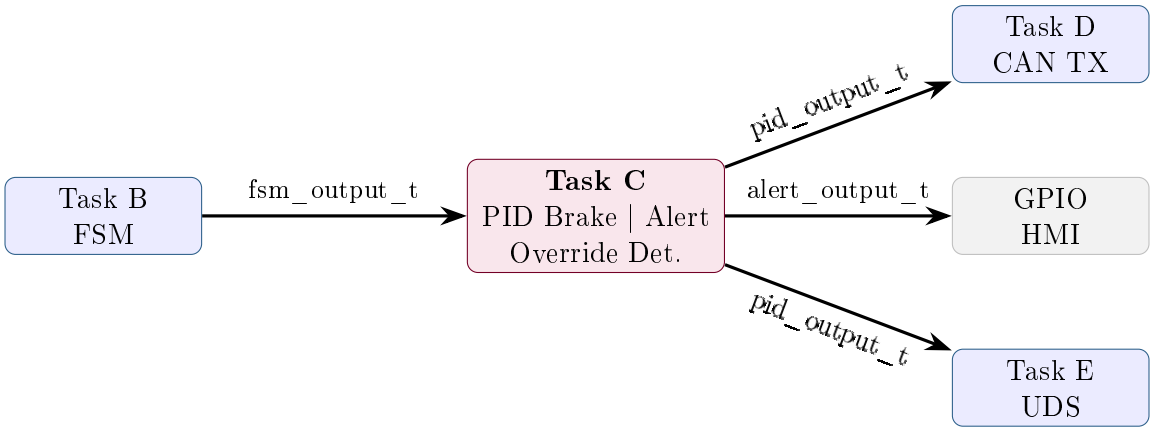

Figure 1: Execution module data flow within the AEB system.

2.2 Simulink Model Reference

The implementation maps directly to three Simulink charts from `AEB_Integration.slx`:

Table 1: Simulink chart to C function mapping.

Simulink Chart	C Function	Description
chart_96 (PID_Logic)	`pid_brake_step()`	PI controller with anti-windup and jerk limiting
chart_79 (Alert_Map)	`alert_map()`	FSM state to alert type and buzzer pattern
chart_88 (Override_Det)	`override_detect()`	Brake pedal and steering override detection

3 Design Choices

3.1 PI Controller with Anti-Windup

The brake controller uses a PI control law with proportional gain $K_p = 10.0$ and integral gain $K_i = 0.05$:

$$e(k) = a_{\text{target}} - a_{\text{ego}}, \qquad I(k) = I(k-1) + K_i \cdot e(k) \cdot \Delta t, \qquad u(k) = K_p \cdot e(k) + I(k) \quad (1)$$

The integral term is clamped to [0, 50%] (anti-windup). The lower bound of zero prevents negative integral accumulation, since the AEB system only brakes — it never commands acceleration.

Design Decision: Integral clamp at zero

Unlike a general-purpose PID, the AEB PI only produces positive brake commands. Allowing negative integral would mean the controller "wants to accelerate," which is the FSM's responsibility (state transition to STANDBY). Clamping at zero ensures clean reset behaviour.

3.2 Jerk Limiting

Longitudinal jerk is limited to $|\dot{a}| \leq 10\,\text{m/s}^3$ (FR-BRK-004). This is converted to a maximum output change per cycle:

$$\Delta u_{\max} = J_{\max} \times \frac{100\%}{a_{\max}} \times \Delta t = 10 \times \frac{100}{6} \times 0.01 \approx 1.667\,\%/\text{cycle} \quad (2)$$

This also satisfies FR-BRK-001 (no jumps $> 2\,\text{m/s}^2$ between cycles), since 1.667% maps to approximately $1.0\,\text{m/s}^2$ per cycle.

3.3 Controller State Reset

The PI integrator and previous output are reset to zero when the FSM leaves braking states (`fsm_state` < `FSM_BRAKE_L1`) or when `decel_target` ≤ 0. This prevents integrator carry-over when the system transitions through WARNING → STANDBY → BRAKE_L1 again.

3.4 Alert Mapping as Pure Combinational Logic

The `alert_map()` function is stateless — a direct lookup from FSM state to alert outputs. The 800 ms minimum warning time (FR-ALR-003) is enforced by the FSM module (Task B), not by the alert module. This keeps the function simple and testable.

3.5 Override Detection Scope

The `override_detect()` function reports the general case: brake pedal OR steering angle $> 5°$. The POST_BRAKE exception (FR-FSM-006: only accelerator triggers override) is handled by the FSM module, which ignores the override signal in that state. This maintains separation of concerns.

Design Decision: No HAL abstraction needed

Unlike Task D (CAN), the Execution module has no hardware dependencies. Both `pid_brake_step()` and `alert_map()` are pure functions that receive structs and produce structs. No stubs directory is needed for host testing.

4 Execution Flow

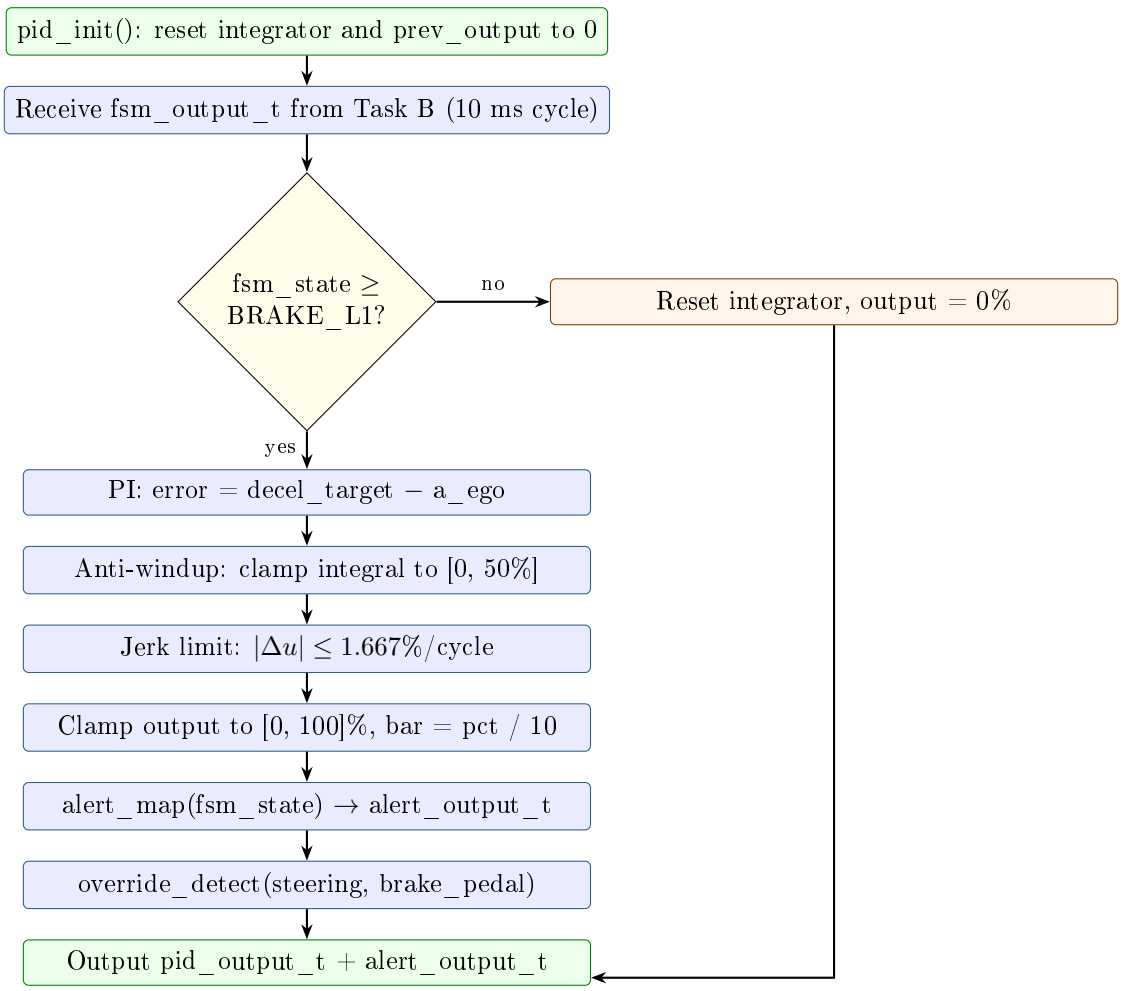

Figure 2: Execution module flow per 10 ms cycle.

5 Requirements Traceability

Table 2: Traceability: SRS requirements → implementation.

Req.	Requirement	Implementation	Test
FR-BRK-001	Progressive decel., no jumps $> 2\,\mathrm{m/s^2}$	Jerk limiter in pid_brake_step()	test_pid_no_abrupt_jumps
FR-BRK-002	PI with anti-windup, error $< 0.5\,\mathrm{m/s^2}$	PI law + integral clamp [0, 50%]	test_pid_steady_state_convergence
FR-BRK-003	Braking $\geq 5.0\,\mathrm{m/s^2}$	Output reaches 100% with integral accumulation	test_pid_max_braking_capability
FR-BRK-004	Jerk $\leq 10\,\mathrm{m/s^3}$	max_delta = 1.667%/cycle	test_pid_jerk_limiting
FR-BRK-005	Maintain brake 2 s after stop	Controller active in POST_BRAKE	test_pid_post_brake_hold
FR-BRK-006	Release on accelerator	FSM transitions to STANDBY, controller resets	test_pid_reset_on_standby
FR-BRK-007	Output [0, 100]%	clamp_f32() after PI and jerk limit	test_pid_output_clamping
FR-ALR-001	Visual alert in WARNING	alert_map() sets ALERT_BOTH	test_alert_warning
FR-ALR-002	Audible alert in WARNING	alert_map() sets buzzer pattern	test_alert_warning
FR-ALR-003	Alert precedes braking $\geq 800\,\mathrm{ms}$	Enforced by FSM (Task B), not this module	N/A (FSM test)
FR-ALR-004	Alert ceases on STANDBY return	alert_map() returns NONE for STANDBY	test_alert_standby
FR-DEC-006	Override on brake pedal	override_detect() checks brake_pedal	test_override_brake_pedal
FR-DEC-007	Override on steering $> 5°$	override_detect() checks abs(angle)	test_override_steering_*

5.1 MISRA C:2012 Compliance

Table 3: MISRA C:2012 rules applied in the implementation.

Rule	How Applied
Fixed-width types	All variables use `uint8_t`, `uint16_t`, `float32_t`
No dynamic alloc.	No `malloc`/`free`; state in `static` module variables
No recursion	Call graph is acyclic
All vars initialised	Every variable has an initialiser at declaration
No dead code	All functions and branches are reachable
Default in switch	`alert_map()` switch includes `default` for invalid states
No goto	No `goto` used
No math.h in prod.	Custom `abs_f32()` helper avoids library dependency

5.2 MISRA Validation

Listing 1: MISRA validation via cppcheck.

```
cppcheck --addon=misra --std=c99 -Iinclude \
  --suppress=unusedFunction --suppress=missingIncludeSystem \
  --enable=all src/aeb_pid.c src/aeb_alert.c
```

Combined with `gcc -Wall -Wextra -Wpedantic -std=c99` (zero warnings on all files).

6 Test Summary

The module includes **45 unit test assertions** across 2 test suites:

Table 4: Unit test results (45 assertions, 45 passed).

Test	What it validates	Result
	test_pid.c — 18 assertions	
test_pid_no_braking_in_standby	Zero output in STANDBY state	PASS
test_pid_no_braking_in_off	Zero output in OFF state	PASS
test_pid_no_braking_in_warning	Zero output in WARNING (alert only)	PASS
test_pid_brake_l1_produces_output	Positive output after 50 cycles in L1	PASS
test_pid_output_clamping	Output saturates within [0, 100]%	PASS
test_pid_jerk_limiting	$\Delta u \leq 1.667\%$/cycle across 100 cycles	PASS
test_pid_steady_state_convergence	Negligible drift $< 2\%$ over 50 cycles	PASS
test_pid_max_braking_capability	Achieves $\geq 5.0\,\mathrm{m/s^2}$	PASS
test_pid_post_brake_hold	Braking continues in POST_BRAKE	PASS
test_pid_reset_on_standby	Integrator clears on STANDBY return	PASS
test_pid_no_abrupt_jumps	No jumps $> 33.3\%$ ($2\,\mathrm{m/s^2}$)	PASS
test_pid_bar_conversion	bar = pct / 10 always	PASS
	test_alert.c — 27 assertions	
test_alert_off	NONE + SILENT in OFF	PASS
test_alert_standby	NONE + SILENT in STANDBY	PASS
test_alert_warning	BOTH + SINGLE in WARNING	PASS
test_alert_brake_l1	BOTH + DOUBLE in L1	PASS
test_alert_brake_l2	BOTH + FAST_PULSE in L2	PASS
test_alert_brake_l3	BOTH + CONTINUOUS in L3	PASS
test_alert_post_brake	NONE + SILENT in POST_BRAKE	PASS
test_alert_invalid_state	Safe default for unknown state	PASS
test_override_none	No override with zero inputs	PASS
test_override_brake_pedal	Override on brake press	PASS
test_override_steering_positive	Override at $+6°$	PASS
test_override_steering_negative	Override at $-7°$	PASS
test_override_steering_boundary	No override at exactly $5°$	PASS
test_override_steering_below	No override at $4.9°$	PASS
test_override_both	Override with both inputs	PASS

References

1. AEB SRS v2.0, "Software Requirements Specification," March 2026.
2. AEB Implementation Strategy v1.0, "AEB_Tasks," April 2026.
3. `AEB_Integration.slx`, Simulink/Stateflow model.
4. MISRA, "MISRA C:2012," 3rd edition, 2019.